

MedAssist AI: Medical Symptom Analysis & Disease Prediction System

Project Title: MedAssist AI: Medical Symptom Analysis & Disease Prediction System

Project Type: AI-Powered Healthcare Assistance Platform

Document Type: Final Project Technical Documentation & Comprehensive Evaluation Report

Implementation Period: 8-Week Lifecycle (Milestones 1 to 4)

Authors: MedAssist AI Development Team

Date: September 2026

Executive Summary

MedAssist AI is an AI-powered healthcare assistance platform designed to provide symptom analysis, preliminary disease prediction, health risk assessment, healthcare recommendations, and automated health report generation. Built using Python (FastAPI + Uvicorn), React.js with Vite, PostgreSQL 15, and Scikit-Learn Machine Learning Pipelines, the system supports healthcare assistance workflows for both patients and authorized caretakers.

The platform integrates two dedicated machine learning engines:

1. **Disease Prediction Engine:** A multi-class RandomForestClassifier trained on the disease-symptom dataset across 131 symptom features and 41 disease categories. The available symptom-disease records were transformed into binary symptom vectors, and additional training samples were generated from verified symptom combinations to expose the classifier to partial symptom presentations and improve robustness when fewer symptoms are provided (achieving 89.23% test accuracy and 89.33% macro F1-score on the evaluated test split). These metrics represent performance on the evaluated dataset/test split and should not be interpreted as real-world clinical diagnostic accuracy.
2. **Health Risk Assessment Engine:** A ColumnTransformer + RandomForestClassifier pipeline evaluating 8 demographic, symptom, and vital indicators (Age, Gender, Blood Pressure, Cholesterol, Fever, Cough, Fatigue, Difficulty Breathing). The model performs binary classification between lower-risk/standard-risk and higher-risk categories, achieving 92.16% test accuracy, 100.0% recall for the high-risk class on the evaluated test dataset (with zero false negatives observed in that test set), and a 0.9769 ROC-AUC score.

The complete platform is containerized using Docker and Docker Compose, validated via automated Pytest unit tests (10/10 passed) and Postman API collection testing, and deployed to Amazon Web Services (AWS) EC2 cloud infrastructure.

Table of Contents

1. [Project Overview & Objectives](#1-project-overview--objectives)
 2. [Problem Statement & Scope](#2-problem-statement--scope)
 3. [System Architecture](#3-system-architecture)
 4. [Technology Stack & Modules](#4-technology-stack--modules)
 5. [Datasets & AI/ML Model](#5-datasets--ai-ml-model)
 6. [Implementation & Milestones](#6-implementation--milestones)
 7. [User Roles & Dashboard](#7-user-roles--dashboard)
 8. [Testing & Performance Evaluation](#8-testing--performance-evaluation)
 9. [Deployment & Results](#9-deployment--results)
 10. [Conclusion & Future Enhancement](#10-conclusion--future-enhancement)
- [References](#references)

1. Project Overview & Objectives

1.1 Project Title & Overview

MedAssist AI is a centralized, AI-driven digital healthcare assistance platform designed to assist users in understanding potential health conditions early, analyzing symptoms, evaluating health risks, receiving evidence-based healthcare recommendations, and facilitating structured continuity of care with authorized caretakers. The platform provides self-reporting patients and outpatient care providers with an integrated web interface for symptom evaluation, differential disease ranking, health risk assessment, dynamic PDF health reporting, and caretaker population analytics.

1.2 Motivation & Healthcare Problem Addressed

Healthcare delivery systems face operational bottlenecks, outpatient congestion, and preliminary intake challenges:

- **Patient Uncertainty:** Individuals experiencing initial symptoms (e.g., fever, headache, nausea, mild fatigue) often struggle to understand their significance, leading either to delayed consultation or unnecessary emergency visits.
- **Search Misinformation:** Generic search queries frequently return alarmist, un-stratified results that induce panic or encourage unguided self-medication.
- **Fragmented Caregiver Oversight:** Caregivers and healthcare assistants often lack centralized mechanisms to track outpatient symptom histories, monitor risk patterns, and share structured care plans remotely.

MedAssist AI addresses these challenges by introducing structured, machine-learning-assisted symptom evaluation, health risk assessment, and advisory support within a secure digital environment.

1.3 Why AI/ML is Used

Traditional symptom checkers rely on rigid rule-based decision trees that fail when patients present atypical, sparse, or misspelled symptom combinations. Machine learning provides:

- **Multi-Disease Ranking:** Generates top-3 predicted disease conditions with associated model-generated probability scores across 41 disease classes simultaneously.
- **Multi-Parameter Risk Assessment:** Simultaneously evaluates age, vitals (blood pressure, cholesterol), and acute symptoms to identify elevated health risk categories.
- **Robustness for Sparse Inputs:** Combinatoric data augmentation trains the model to maintain predictive reliability even when users disclose only partial symptom subsets.

1.4 Core Objectives & Expected Outcomes

ID	Objective Area	Implementation Deliverable
OBJ-1	Symptom Analysis	Standardize 131 symptom features with typo and synonym handling
OBJ-2	Disease Prediction	Multi-class Random Forest model predicting 41 diseases with probability scores
OBJ-3	Health Risk Assessment	8-parameter pipeline evaluating risk classification, recall, and ROC-AUC
OBJ-4	Healthcare Guidance	Curated precautions, lifestyle, and dietary recommendations for 41 conditions
OBJ-5	Health Reports	Automated PDF health summaries for patients and authorized caretakers
OBJ-6	Caretaker Analytics	Interactive Recharts visualizations and caretaker care-plan management
OBJ-7	Cloud Deployment	Multi-container Docker stack deployed to AWS EC2 infrastructure

1.5 Target Users & Benefits

- Patients: Immediate preliminary symptom analysis, risk awareness, and downloadable PDF health summaries.
- Caretakers & Healthcare Providers: Cohort tracking, risk distribution analytics, and structured care-plan management.
- Outpatient Clinics: Digital pre-consultation intake, symptom recording, and structured historical dossiers.

2. Problem Statement & Scope

2.1 Problem Statement & Limitations of Traditional Workflows

Manual symptom intake in primary healthcare is often unstructured, paper-based, and subjective. Clinical staff spend valuable consultation time gathering basic demographic and symptom baselines that can be pre-recorded digitally. Furthermore, patients lack a persistent digital record of their previous predictions, risk assessments, and caretaker directives.

2.2 Proposed Solution & System Scope

MedAssist AI provides an automated digital symptom analysis and advisory workflow:

- **Intake:** Patient registers and enters present symptoms with severity levels and vital indicators.
- **AI Processing:** ML models classify disease candidates and compute health risk assessment categories.
- **Output & Insights:** Displays top-3 predicted disease conditions with model-generated probability scores, descriptions, precautions, and personalized lifestyle/dietary guidance.
- **Caretaker Oversight:** Assigned caretakers review records, analyze population trends, and manage care plans.

2.3 In-Scope vs. Out-of-Scope Boundaries

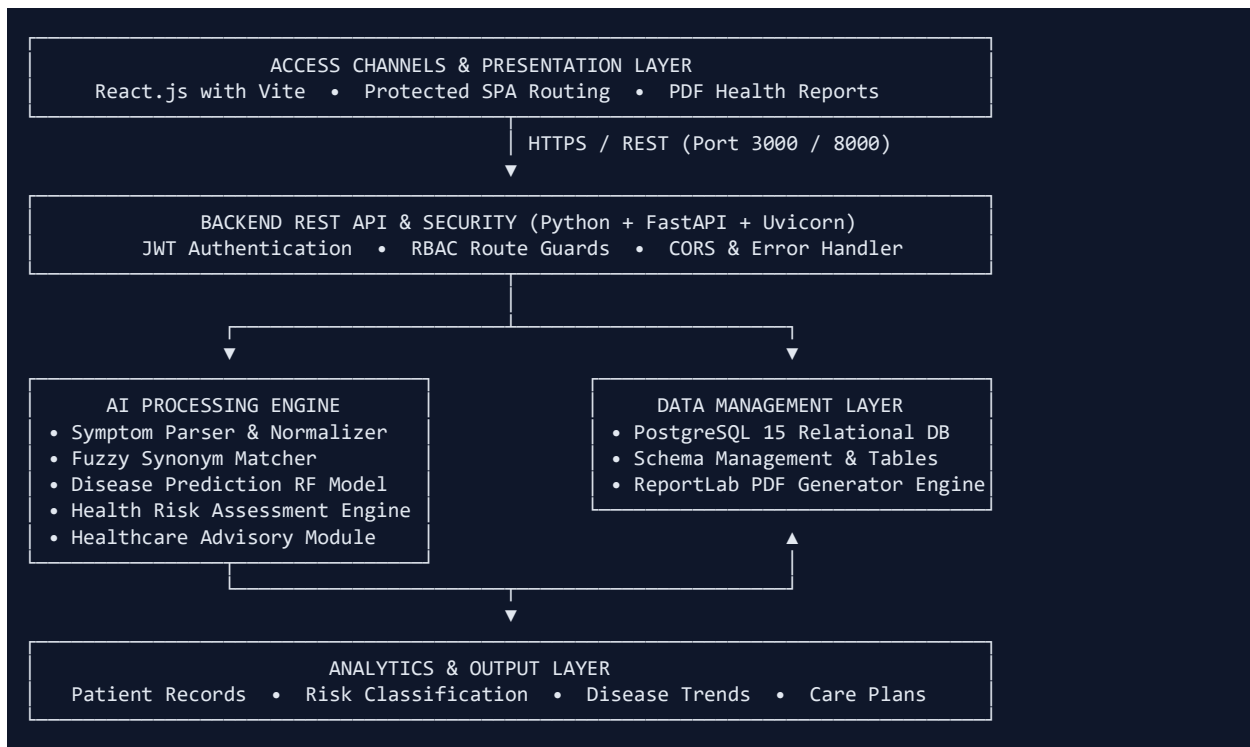
Functional Area	In-Scope (Implemented & Verified)	Out-of-Scope (Future Enhancements)
Disease Prediction	41 disease categories, 131 symptom inputs, top-3 ranked predictions	Rare genetic disorders, oncology subtyping
Risk Assessment	8 demographic and vital indicators (Age, BP, Cholesterol, etc.)	Real-time continuous ICU waveform monitoring
User Roles	Patient and Caretaker with RBAC authorization	Insurance billing agents, laboratory technicians
Advisory & Reports	41 curated health recommendations, ReportLab PDF download	Direct pharmacy prescription ordering
Analytics	Caretaker cohort charts (Recharts bar, pie, line)	Hospital-wide bed occupancy forecasting
Deployment	Docker, Docker Compose, AWS EC2 Cloud VM	Distributed multi-region Kubernetes clusters

3. System Architecture

3.1 Overall Architecture & Layered Design

The platform is organized into 5 modular tiers:

- Access Channels & Presentation Tier (React.js with Vite): Single Page Application with high-contrast UI, responsive form components, Recharts visualizations, and PDF downloads.
- Backend REST API & Security Layer (Python + FastAPI + Uvicorn): Asynchronous REST endpoints, JWT authentication (24h expiry), bcrypt password hashing (with 72-byte truncation safety), and RBAC route guards.
- AI Processing Engine: 131-dimensional binary feature vectorizer, multi-class Random Forest disease model, ColumnTransformer risk assessment pipeline, and difflib fuzzy synonym matcher.
- Data Management Layer (PostgreSQL 15): 9 relational tables (`users`, `patient\_profiles`, `caretaker\_profiles`, `patient\_symptoms`, `disease\_predictions`, `patient\_risk\_assessments`, `patient\_reports`, `patient\_assignments`, `caretaker\_care\_plans`).
- Analytics & Output Layer: Cohort risk distributions, disease trends, PDF generator, and care plans.



3.2 Patient Healthcare Assistance Workflow

[IMPORTANT] Patient Workflow Sequence: Login → Patient Dashboard → Symptom Selection/Input → Symptom Severity → Disease Prediction → Risk Assessment → Healthcare Recommendations → PDF Health Report → Assessment History

The selected symptoms are converted into the feature representation expected by the trained model. The Random Forest classifier returns predicted disease classes with model-generated probability scores. The application presents top-ranked predictions to the user as preliminary informational results rather than a definitive medical diagnosis.

3.3 Caretaker Workflow

[IMPORTANT] Caretaker Workflow Sequence: Login → Caretaker Dashboard → View Authorized Patient Information → Review Assessments/Analytics → Select Patient → Create or Update Care Plan → Save/Manage Care Plan

4. Technology Stack & Modules

4.1 Implemented Technology Stack Matrix

Category	Technology	Role in MedAssist AI
Frontend Framework	React.js with Vite	Single Page Application & UI components
Routing	React Router DOM	SPA client-side routing & route guards
Data Visualization	Recharts	Interactive Bar, Pie, and Line charts
UI Iconography	React Icons	Dashboard iconography
Backend Language	Python	Core API runtime & ML execution
Backend Framework	FastAPI	Asynchronous REST API framework
ASGI Server	Uvicorn	High-performance ASGI web server
Data Validation	Pydantic	Request schema parsing & validation
Machine Learning	Scikit-Learn	Random Forest, Preprocessing Pipelines
Data Processing	Pandas & NumPy	Feature matrices & data structures
Database	PostgreSQL	Relational database for health records
Authentication	Passlib (Bcrypt) + PyJWT	Safe password hashing & JWT tokens
PDF Generation	ReportLab	Binary PDF health report creation
Containerization	Docker & Compose	Multi-container stack orchestration
Web Server	Nginx	Reverse proxy & static SPA hosting
Cloud Hosting	AWS EC2	Cloud VM hosting (t2.micro)
Testing	Pytest & Postman	Automated unit API testing

4.2 The 7 Core Implemented Modules

- User Management Module (auth.py, database.py): Registration (/auth/register), authentication (/auth/login), bcrypt password security with safe 72-byte truncation, patient profile CRUD, and caretaker profile CRUD.
- Symptom Analysis Module (prediction_service.py, patient.py): 131-symptom catalog lookup, severity tracking, semantic synonym dictionary, and difflib fuzzy string matching.
- Disease Prediction Module (prediction_service.py, train_disease_model.py): Multi-class Random Forest disease classification (/patient/predict-disease), probability scores, and top-3 ranked condition outputs.
- Risk Assessment Module (patient_risk_service.py, train_patient_risk_model.py): 8-parameter pipeline evaluating risk probability and categorical risk classification.
- Healthcare Recommendation Module (treatment_data.py, patient.py): Curated precautions, dietary guidance, lifestyle advice, and recommendations to seek medical consultation across all 41 conditions.
- Health Reports Module (report_generator.py, patient.py): Automated ReportLab PDF generator compiling demographics, symptoms, predictions, risk scores, and guidance.
- Analytics Dashboard Module (analytics.py, caretaker.py, CaretakerAnalytics.jsx): Caretaker cohort analytics visualizing disease distributions, risk ratios, and consultation trends.

5. Datasets & AI/ML Model

5.1 Dataset Overview: Implemented vs. Recommended Datasets

The project specification recommended multiple healthcare datasets. For the implemented workflow, primary symptom-disease datasets were used for model training. Other datasets mentioned in the specification represent recommended or potential datasets rather than directly connected components of the final codebase:

Dataset Name	Status	Usage Details in MedAssist AI
Disease-Symptom Mapping Dataset (<code>dataset.csv</code> , <code>symptom_*.csv</code>)	IMPLEMENTED	Provides symptom-to-disease mappings for 41-class disease prediction.
Patient Profile & Risk Dataset (<code>patient_profile_dataset.csv</code>)	IMPLEMENTED	Provides 8 demographic/vital inputs for binary health risk modeling.
CDC BRFSS Dataset	RECOMMENDED / NOT IMPLEMENTED	Recommended dataset option for risk analysis extensions; not in pipeline.
MIMIC-IV Dataset	RECOMMENDED / NOT IMPLEMENTED	Optional advanced clinical dataset; not required for student workflow.

5.2 Disease Model Formulation & Training

The disease prediction task is formulated as a 41-class classification problem over 131 binary symptom features. The symptom-disease records were converted into binary vectors and expanded via combinatoric subset augmentation across each disease class, creating a structured corpus of 4,502 augmented training samples to enhance model robustness for partial symptom disclosures.

- **Model Architecture:** RandomForestClassifier (100 decision trees, Gini impurity, balanced class weights).
- **Probability Estimates:** Tree voting probabilities are sorted to return the top-3 preliminary candidate conditions.
- **Synonym & Typo Resolver:** Medical synonym dictionary (SYMPTOM_ALIASES) combined with `difflib.get_close_matches(cutoff=0.8)`.

Table 5.1: Disease Prediction Model Performance Metrics

Metric Parameter	Evaluated Value	Description
Model Algorithm	RandomForestClassifier (100 Trees)	Ensemble tree voting
Disease Classes Covered	41 Classes	Multi-class classification problem
Total Input Features	131 Symptoms	Binary symptom indicators
Augmented Training Corpus	4,502 Samples	Stratified combinatoric subsets
Test Accuracy (Stratified 20%)	89.23% (0.8923)	Performance on evaluated test split
Macro F1-Score	89.33% (0.8933)	Balanced multi-class precision/recall
Overall Dataset Accuracy	89.36% (0.8936)	Holistic classification performance
5-Fold Cross Validation Mean	85.98% (0.8598)	Cross-validation accuracy across folds
5-Fold Cross Validation Std	$\pm$ 0.51% (0.0051)	Variance across evaluated folds

Table 5.2: Symptoms with Highest Feature Importance in the Trained Model

Rank	Feature Name	Gini Importance	Condition Relevance in Dataset
1	vomiting	2.15%	High discriminative factor in training dataset
2	fatigue	2.05%	Common systemic marker across conditions
3	high_fever	1.84%	Acute febrile illness indicator
4	nausea	1.65%	Gastrointestinal symptom indicator
5	loss_of_appetite	1.61%	Systemic illness indicator
6	headache	1.48%	Febrile and general indicator
7	chest_pain	1.43%	Cardiovascular and respiratory indicator
8	itching	1.32%	Dermatological symptom indicator
9	joint_pain	1.30%	Musculoskeletal and viral indicator

5.3 Patient Risk Model Formulation & Performance

The patient risk model performs binary classification between lower-risk/standard-risk and higher-risk categories based on 8 explicit features: Age (continuous, StandardScaler), Gender (categorical), Blood Pressure (Normal/High/Low), Cholesterol Level (Normal/High), Fever (Binary), Cough (Binary), Fatigue (Binary), and Difficulty Breathing (Binary).

Table 5.3: Patient Risk Assessment Model Performance Metrics

Metric Parameter	Evaluated Value	Significance
Pipeline Architecture	ColumnTransformer + RandomForest	Scaled continuous + one-hot encoded categorical
Test Accuracy	92.16% (0.9216)	Performance on evaluated test split
Balanced Accuracy	92.00% (0.9200)	Balanced performance across risk classes
Precision (High Risk)	86.67% (0.8667)	Precision on positive risk predictions
Recall (High Risk)	100.0% (1.0000)	100% recall for high-risk class on test set
F1-Score	92.86% (0.9286)	Harmonic mean of precision and recall
ROC-AUC Score	97.69% (0.9769)	Area under ROC curve on test split

Table 5.4: Patient Risk Model Confusion Matrix

Actual Class \ Predicted Class	Predicted Low Risk	Predicted High Risk	Total
Actual Low Risk	21 (True Negative)	4 (False Positive)	25
Actual High Risk	0 (False Negative)	26 (True Positive)	26
Total	21	30	51

(Note on Sensitivity: On the evaluated test set, the high-risk class achieved 100% recall with zero false negatives. This is an evaluation result on the test split and does not guarantee detection of all high-risk clinical cases in real-world settings).

6. Implementation & Milestones

6.1 Lifecycle & Milestone Summary Table

The development followed an 8-Week Engineering Lifecycle across 4 two-week milestones:

Milestone	Target Weeks	Major Implemented Work	Verified Outcome
Milestone 1	Weeks 1 & 2	System architecture, database schema, JWT authentication, RBAC, symptom data preparation.	Functional RBAC authentication and database persistence.
Milestone 2	Weeks 3 & 4	Disease prediction models, probability scoring, patient risk classification, PDF health summaries.	Working symptom prediction and risk calculation workflows.
Milestone 3	Weeks 5 & 6	Healthcare recommendations, advisory guidance, caretaker dashboard, Recharts analytics, caretaker care-plan management.	Caretaker health tracking, patient assignments, and health trends visualization.
Milestone 4	Weeks 7 & 8	Model accuracy optimization, typo resolver, UI contrast overhaul, Care Plans module, Pytest suite (10/10 passed), Dockerization, AWS EC2 cloud deployment.	The completed application was containerized and deployed to an AWS cloud environment for end-to-end demonstration and validation.

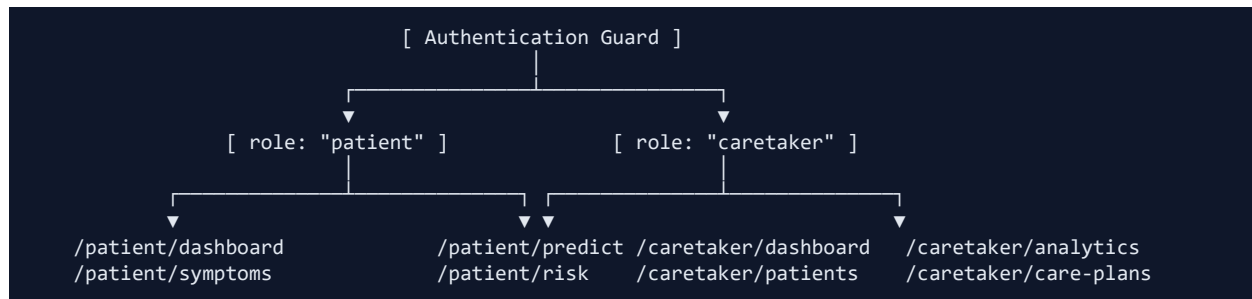
6.2 Milestone Highlights

- **Milestone 1:** Planned primary user workflows, initialized FastAPI and React.js with Vite repositories, built PostgreSQL schemas, and implemented bcrypt/JWT authentication.
- **Milestone 2:** Implemented Random Forest disease prediction and ColumnTransformer risk assessment pipelines; built ReportLab PDF generation engine.
- **Milestone 3:** Mapped 41 conditions to curated healthcare recommendations; built Caretaker Dashboard, patient rosters, Recharts analytics, and care-plan management.
- **Milestone 4:** Combinatoric dataset augmentation (4,502 samples), typo resolver, high-contrast UI overhaul, automated Pytest suite (10/10 passed), Docker orchestration, and AWS EC2 deployment.

7. User Roles & Dashboard

7.1 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) ensures that authenticated users access functionality according to their assigned role. Patient and caretaker workflows are separated so that patient information and caretaker functionality are accessed through the appropriate application paths.



7.2 Patient Portal & Dashboard Workflows

Patient Role: Can register/login, maintain a profile, enter symptoms and symptom severity, run disease prediction and health-risk assessment, view healthcare recommendations, review assessment history, and generate/download health reports. The patient does not receive a formal medical diagnosis; predictions are delivered as preliminary informational results.

- **Profile Management:** View and update demographics, blood type, height, weight, and emergency contacts.
- **Symptom Selection & Severity:** Choose symptoms from the 131-symptom catalog and indicate severity (Mild, Moderate, Severe).
- **Preliminary Disease Prediction:** Converts symptoms into feature vectors; Random Forest returns top-3 predicted diseases with probability scores.
- **Health Risk Assessment:** Evaluates vital indicators to determine risk probability and classification.
- **Healthcare Recommendations:** Disease-specific precautions, dietary advice, lifestyle guidance, and advice to seek professional evaluation.
- **Health Reports & History:** Historic prediction review and downloadable structured PDF health summaries.

7.3 Caretaker Dashboard Workflows

Caretaker/Healthcare Provider Role: Can access authorized patient information, review patient assessments, view available analytics, and create/manage care-plan information and health guidance for assigned patients.

- **Authorized Patients Directory:** Roster of linked patients and profile summaries.

- **Patient Health Records:** Review historic symptoms, prediction logs, vital indicators, and risk classification records.
- **Health Analytics & Trends** (/caretaker/analytics):
- **Top Conditions Bar Chart:** Frequency of predicted diseases across assigned patients.
- **Risk Classification Pie Chart:** Proportion of High, Medium, and Low risk cases.
- **Consultation Timeline Chrt:** Monthly consultation activity trends over time.
- **Caretaker Care Plans** (/caretaker/care-plans): Formulate care plans with priority tags (Urgent, High, Standard, Follow-up), recording observations, health guidance, and dietary/lifestyle recommendations.

8. Testing & Performance Evaluation

8.1 Quality Assurance Strategy

Quality assurance combined automated unit testing via Pytest, API testing via Postman, and empirical model validation.

8.2 Test Cases Execution Summary Table (10/10 Passed)

Test ID	Component	Input Test Case	Expected Behavior	Actual Result	Status
TC-01	prediction_service	Feature Names List	Exactly 131 features loaded in memory	131 features verified	PASSED
TC-02	prediction_service	['itching', 'skin_rash']	Valid disease string returned	Returns 'Fungal infection'	PASSED
TC-03	prediction_service	predict_top_conditions(5 syms)	Returns 3 objects with scores in [0, 100]	3 valid objects with scores	PASSED
TC-04	prediction_service	get_disease_precautions('Diabetes')	Returns non-empty precautions list	Returns 4 precautions	PASSED
TC-05	patient_risk_service	High-risk 65yo patient	predicted_outcome: 'Positive', score >= 70%	Outcome 'Positive', score 75.0%	PASSED
TC-06	patient_risk_service	Incomplete vital dictionary	Raises validation error	Raises ValueError("Missing required fields")	PASSED
TC-07	treatment_data	Exact match 'Diabetes'	Metabolic suggestions >= 3 items	Category 'Metabolic', 4 suggestions	PASSED
TC-08	treatment_data	Unmapped disease string	Graceful fallback suggestions	Category 'General' with safety advice	PASSED
TC-09	treatment_data	All 41 disease coverage	All 41 diseases have curated advice	41/41 diseases verified	PASSED
TC-10	report_generator	Full patient record	Non-empty binary PDF starting with b'%PDF'	Valid PDF bytes generated (>500B)	PASSED

8.3 Postman API Collection & System Observations

A pre-configured Postman collection (MedAssist_AI_Postman_Collection.json) evaluates backend endpoints across Authentication, AI Predictions, Patient Records, Caretaker Modules, and System Health. The system was functionally validated for API responsiveness, model inference latency, and PDF generation.

9. Deployment & Results

9.1 Containerization with Docker & Docker Compose

MedAssist AI implements a 3-tier container architecture defined in ``docker-compose.yml``:

- **``medassist_frontend``**: React 18 production build served via Nginx 1.25 Alpine on port ``3000 -> 80``.
- **``medassist_backend``**: FastAPI application served via Uvicorn on port ``8000 -> 8000``.
- **``medassist_db``**: PostgreSQL 15 Alpine database with persistent volume ``postgres_data`` on port ``5432 -> 5432``.

9.2 Cloud Deployment on AWS EC2

- **Instance**: ``t2.micro`` (1 vCPU, 1GB RAM), Ubuntu Server 24.04 LTS.
- **Virtual Memory Configuration**: Established a 2GB swap partition (``/swapfile``) on the 30GB gp3 root volume to ensure smooth container execution and memory headroom.
- **Security Group Inbound Ports**: ``22`` (SSH), ``80/443`` (HTTP/HTTPS), ``3000`` (Frontend Portal), ``8000`` (Backend API & Swagger Docs).
- **Deployment Verification**: Verified end-to-end on AWS EC2 with all 3 containers running with status ``Up`` and ``healthy``.

10. Conclusion & Future Enhancement

10.1 Project Conclusion & Key Achievements

The MedAssist AI project successfully implemented the core functional objectives defined for the academic project across all four planned milestones:

- Built a 41-class Random Forest disease prediction model with 89.23% test accuracy and combinatoric augmentation for sparse-input robustness.
- Implemented an 8-parameter health risk assessment pipeline with 92.16% test accuracy, 100% recall for high-risk test cases (with zero false negatives observed in that test set), and a 0.9769 ROC-AUC score.
- Developed semantic synonym and fuzzy string matching utilities using Python's `difflib`.
- Implemented an interactive Caretaker Dashboard with Recharts visual analytics and caretaker care-plan management.
- Containerized the full-stack system and deployed it to Amazon Web Services (AWS) EC2 cloud infrastructure.

10.2 Technical Limitations & Future Roadmap

- Current Limitations: Tabular feature inputs rather than unstructured EHR notes; absence of continuous IoT vital streaming; English medical nomenclature only.
- Future Enhancements:
 - EHR / FHIR Interoperability: Synchronization with hospital Electronic Health Record systems via HL7 FHIR standards.
 - Conversational LLM Intake: Voice and conversational chat-based symptom intake.
 - Explainable AI (XAI): SHAP / LIME feature attribution visualizations in the dashboard.
 - Wearable IoT Stream Ingestion: Vital synchronization from consumer health tracking devices.
 - Multilingual Medical Localization: Expanding the synonym dictionary to regional languages.

References

1. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.
2. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
3. Ramirez, T. (2020). *FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python 3.8+*.
4. Kaggle Healthcare Datasets: *Disease Symptoms and Patient Profile Dataset* and *Disease Prediction Using Symptoms Dataset*.
5. Centers for Disease Control and Prevention (CDC). (2023). *Behavioral Risk Factor Surveillance System (BRFSS)*.
6. World Health Organization (WHO). (2023). *Digital Health Guidelines and Clinical Decision Support Standards*.
7. Postman Inc. (2026). *Automated API Testing and Collection Runner Documentation*.
8. Amazon Web Services (AWS). (2026). *Amazon EC2 User Guide for Linux Instances & Container Architectures*.